MINI-PROJET POO · SUJET 2

Moteur de combat tactique : l'Arène des Légendes

Cahier des charges pédagogique

Wissem Zeddini · À lire avec le document « Mini-projets POO : cadre commun »

Client	M. Yassine Mrad, directeur de production du studio Pixel Dunes (client simulé)
Le logiciel	Le moteur logique d'un jeu de rôle au tour par tour : des héros et des monstres s'affrontent dans une arène
Difficulté	Intermédiaire (2 sur 3), avec un défi métier plus exigeant
Notions du socle	Encapsulation, constructeurs, héritage, redéfinition, polymorphisme
Bonus possibles	Abstraction (badge Abstrait), exception personnalisée (badge Résilient), sauvegarde dans un fichier (badge Archiviste)
Livrable final	Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes

1. Le contexte

Le studio **Pixel Dunes** (un studio fictif) fabrique un jeu de rôle au tour par tour. Les graphistes dessineront plus tard les personnages ; il manque le **moteur de combat** : la logique qui décide qui frappe qui, combien de points de vie il reste, et ce qui se passe quand un héros est empoisonné ou un monstre étourdi.

Vous êtes l'équipe du moteur. Il n'y a ni écran graphique ni son : seulement des objets qui se battent, affichés dans la console. Le client changera d'avis plusieurs fois (nouveaux héros, nouveaux pouvoirs) : concevez dès le départ pour que votre code puisse évoluer.

2. Le vocabulaire du métier

Terme	Sens pour le client
Combattant	Un personnage qui peut attaquer et recevoir des coups : un héros ou un monstre.
Points de vie (PV)	La santé d'un combattant, de 0 jusqu'à son maximum (PV max).
Armure	Une valeur qui réduit les dégâts reçus.
K.O.	Un combattant à 0 PV : il est hors combat.
Mana	L'énergie magique du mage ; chaque sort en consomme.
Flèches	Les munitions de l'archer.
Compétence spéciale	Un pouvoir propre à chaque type de combattant, utilisé tous les 3 tours.
Tour de jeu	Le moment où un combattant agit : il attaque ou utilise sa compétence.
Duel	Un combat entre deux combattants, tour par tour, jusqu'au K.O. d'un des deux.
Effet d'état	Un état temporaire : poison, étourdissement ou bouclier.
Boss	Un monstre très puissant, ajouté plus tard dans le jeu.

3. Les règles du métier

Ces règles sont celles du client. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

N°	Règle	Apparaît en
R1	Les PV d'un combattant restent **toujours entre 0 et ses PV max**.	v0.2
R2	Les dégâts reçus valent **le montant du coup moins l'armure, avec un minimum de 1** : un coup fait toujours au moins 1 dégât.	v0.2
R3	Un combattant **K.O.** ne subit plus de coups, ne peut plus être soigné et ne joue plus.	v0.2
R4	Chaque type de combattant a **sa propre attaque de base** (tableau ci-dessous).	v0.3
R5	Le **mage** dépense du mana à chaque sort ; sans assez de mana, il donne un coup de bâton.	v0.3
R6	L'**archer** dépense des flèches ; sans flèche, il se bat au corps à corps.	v0.3
R7	Le **poison** retire des PV au début de chaque tour pendant plusieurs tours, **sans tenir compte de l'armure**. L'**étourdissement** fait passer le tour du combattant.	v1.0
R8	Un **bouclier** absorbe les dégâts avant que les PV soient touchés.	v1.0
R9	Tous les **3 tours**, un combattant utilise sa compétence spéciale au lieu de son attaque de base.	v1.0

Les types de combattants

Type	PV max	Armure	Attaque de base	Compétence spéciale
Combattant (classe mère)	variable	variable	10	Aucune
Guerrier	120	8	15	Coup de bouclier : 10 dégâts et la cible est étourdie 1 tour
Mage	70	2	Boule de feu : 25 (10 mana), sinon bâton : 5	Nuée toxique (15 mana) : poison de 3 PV par tour pendant 3 tours. Mana de départ : 50
Archer	90	4	Flèche : 18 (1 flèche), sinon corps à corps : 4	Tir précis (2 flèches) : 30 dégâts qui ignorent l'armure. Flèches de départ : 10
Monstre boss (v1.0)	300	10	20	Rugissement : la cible est étourdie 1 tour et le boss gagne un bouclier de 20

*Utilisez des **constantes nommées** plutôt que des nombres écrits au milieu du code. Les PV et les dégâts sont des **entiers**.*

4. Version v0.1 : Cadrage

Mail du client · v0.1

De : Yassine Mrad, studio Pixel Dunes

Objet : Le moteur de notre futur jeu

Bonjour,

Nous construisons un jeu de rôle au tour par tour et il nous faut son moteur de combat. Avant de coder, montrez-moi que vous avez compris : quels sont les objets de mon jeu, ce que chacun sait de lui-même, et affichez-moi un premier combattant à l'écran. La console suffit largement.

Cordialement.

N°	Demande du client	Notion
1.1	« En tant que directeur de production, je veux que vous identifiiez les objets de mon jeu, afin que nous parlions le même langage. »	Analyse, UML
1.2	« Je veux voir un combattant à l'écran, afin de vérifier que mon besoin est compris. »	Classe, constructeur, main

Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple Combattant, Arene et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche un combattant, par exemple : `Thorin · 120/120 PV · armure 8`.

Indice

Un combattant connaît au moins son **nom**, ses **PV**, ses **PV max** et son **armure**. Les types (guerrier, mage, archer) arrivent avec le mail de la version v0.3 : ne les devancez pas.

5. Version v0.2 : Objets fiables

Mail du client · v0.2

De : Yassine Mrad

Objet : Des combattants qui trichent

Bonjour,

Merci pour votre première version. Mes testeurs ont trouvé des bugs étranges : un héros créé avec des PV max négatifs, un monstre mort qui continue à recevoir des coups, et même un cadavre qui se laisse soigner ! Et un coup faible contre une grosse armure… **soignait** le combattant !

Mes règles sont simples : les PV restent entre 0 et le maximum, l'armure réduit les dégâts mais un coup fait toujours au moins 1 dégât, et un combattant K.O. ne participe plus à rien. Personne ne doit pouvoir modifier les PV directement.

Cordialement.

N°	Demande du client	Notion
2.1	« Je veux que les PV restent toujours entre 0 et le maximum, afin d'éviter des combattants absurdes. » (R1)	Encapsulation, validation
2.2	« Je veux qu'un coup fasse (montant moins armure) dégâts, au minimum 1, afin que l'armure compte sans rendre invincible. » (R2)	Méthode métier
2.3	« Je veux soigner un combattant sans dépasser ses PV max, afin de refléter la réalité. »	Méthode métier
2.4	« Je veux qu'un combattant K.O. ne subisse plus rien et ne puisse plus être soigné, afin de respecter les règles. » (R3)	État d'un objet
2.5	« Je veux créer un combattant avec son nom, ses PV max et son armure (PV pleins au départ), afin de gagner du temps. »	Constructeur validé
2.6	« Je veux lire un combattant en une ligne, afin de contrôler vite. »	toString()

Vous livrez

- Tous les attributs **privés**, et **aucun setter pour les PV** : seules les méthodes de coup et de soin les modifient.
- Un constructeur qui **valide** ses paramètres (PV max strictement positifs, armure positive ou nulle).
- Une **Demo** avec au moins trois combattants, qui montre des **refus** : coup sur un K.O., soin d'un K.O., valeur négative.

> **Indices**
>
> Testez les **valeurs limites** : coup de 0, coup négatif, coup plus faible que l'armure, coup plus fort que les PV restants, soin au-delà du maximum. `Math.max` et `Math.min` gardent une valeur entre deux bornes. Pensez à valider aussi dans le constructeur.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
PV max invalide (-5) : 1 utilisé.
Thorin · 120/120 PV · armure 8
Gobelin · 40/40 PV · armure 0
Bug · 1/1 PV · armure 3
  Thorin subit 22 dégâts (98/120 PV)
Thorin · 120/120 PV · armure 8
  Thorin subit 1 dégâts (119/120 PV)
  Gobelin subit 100 dégâts (0/40 PV)
  Gobelin est K.O. !
  Gobelin est déjà K.O. : coup ignoré.
  Un combattant K.O. ne peut pas être soigné.
  Dégâts invalides : ils doivent être positifs.
```

6. Version v0.3 : La famille

Mail du client · v0.3

De : Yassine Mrad

Objet : Nos premiers héros

Bonjour,

Bravo, mes testeurs ne trichent plus. Passons aux héros ! Je veux trois types : un **guerrier** costaud, un **mage** qui lance des sorts avec du mana, et un **archer** qui tire des flèches. Chacun attaque à sa façon (voir le tableau).

Je veux une **arène** où l'on peut lancer l'attaque d'un combattant sur un autre **sans que l'arène sache qui est un guerrier, un mage ou un archer**, et afficher toute l'équipe d'un coup.

Prévenez-moi : le **boss** arrive dans quelques semaines, puis d'autres héros. **Je ne veux pas que vous réécriviez l'arène à chaque nouveau type.**

Cordialement.

N°	Demande du client	Notion
3.1	« Je veux trois types de héros avec leurs particularités (mana, flèches), afin de décrire mes personnages. »	Héritage, super(...)
3.2	« Je veux que chaque type attaque à sa façon, avec ses ressources, afin d'avoir des styles de jeu différents. » (R4, R5, R6)	Redéfinition, @Override
3.3	« Je veux une arène qui lance l'attaque d'un combattant sur un autre sans connaître leur type, afin de la garder simple. »	Polymorphisme
3.4	« Je veux afficher toute l'équipe en une fois, afin de suivre l'état de mes héros. »	Une seule boucle, toString()
3.5	« Je veux ajouter un nouveau type plus tard sans modifier l'arène, afin que mon moteur évolue. »	Preuve du polymorphisme

Vous livrez

- Une **classe mère** qui porte ce qui est commun (PV, armure, coups reçus, soins), et **trois classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Une **Arène** dont la méthode d'attaque prend deux combattants du **type commun**.
- La preuve que l'arène n'a pas besoin d'être modifiée pour accueillir un nouveau type.

Indices

Si vous écrivez **instanceof** dans l'arène, c'est que le polymorphisme manque : quelle méthode chaque type devrait-il redéfinir ? Le mage et l'archer ont chacun une **ressource** (mana, flèches) : qui doit la protéger, et comment ?

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Guerrier Thorin · 120/120 PV · armure 8
Mage Merlin · 70/70 PV · armure 2 · mana 50
Archer Legolas · 90/90 PV · armure 4 · flèches 10
--- Chacun attaque le mannequin ---
Thorin frappe Mannequin.
  Mannequin subit 15 dégâts (185/200 PV)
Merlin lance une boule de feu (mana 40).
  Mannequin subit 25 dégâts (160/200 PV)
Legolas tire une flèche (9 restantes).
  Mannequin subit 18 dégâts (142/200 PV)
Mannequin · 142/200 PV · armure 0
```

7. Version v1.0 : Livraison

> **Mail du client · v1.0**
>
> **De :** Yassine Mrad
>
> **Objet :** Dernière ligne droite avant la démo aux investisseurs
>
> Bonjour,
>
> L'arène est propre, les héros ont du caractère. Pour la démonstration devant nos investisseurs, je veux des combats **vivants**. Je veux des **duels automatiques** tour par tour. **Tous les 3 tours**, chaque combattant doit sortir sa **compétence spéciale**. Et surtout, des **effets d'état** : le poison qui ronge, l'étourdissement qui fait passer un tour, le bouclier qui absorbe les coups.
>
> Le **boss** est enfin prêt à entrer en scène. Rappel : l'arène ne doit pas changer.
>
> Un menu console pour mes testeurs, et un fichier que je puisse lancer moi-même.
>
> Cordialement.

N°	Demande du client	Notion
4.1	« Je veux des duels automatiques, tour par tour, jusqu'au K.O. d'un combattant ou jusqu'à un nombre de tours maximal, afin de tester l'équilibre. »	Boucle de jeu
4.2	« Je veux que chaque combattant utilise sa compétence spéciale tous les 3 tours, afin de varier les combats. » (R9)	Redéfinition, polymorphisme
4.3	« Je veux le poison, l'étourdissement et le bouclier, afin que les combats soient tactiques. » (R7, R8)	Défi métier, état d'un objet
4.4	« Je veux ajouter le boss sans modifier l'arène, afin de prouver que mon moteur évolue. »	Preuve du polymorphisme
4.5	« Je veux un menu console, afin que mes testeurs lancent des combats sans toucher au code. »	Menu, boucle de saisie
4.6	« Je veux un fichier que je lance moi-même, afin de le montrer sur mon poste. »	Fichier .jar

Le défi métier : les effets d'état

Au **début de chaque tour**, avant de jouer, un combattant subit ses effets :

1. le **poison** lui retire des PV (sans tenir compte de l'armure), et un poison de 3 tours s'éteint ensuite ;

2. s'il est **K.O.** à cause du poison, son tour s'arrête là ;

3. s'il est **étourdi**, il passe son tour (l'étourdissement dure un tour) ;

4. le **bouclier**, lui, agit pendant les coups : il absorbe des dégâts avant que les PV soient touchés.

L'arène **ne demande jamais** « es-tu empoisonné ? » : c'est le combattant qui connaît son état.

Menu console attendu (exemple) :

```
=== ARÈNE DES LÉGENDES ===
1. Afficher les combattants
2. Lancer un duel
3. Soigner un combattant
4. Créer un combattant
0. Quitter
Votre choix (0 à 4) :
```

Exemple de sortie d'un duel de 4 tours entre un mage et le boss (la vôtre peut différer) :

```
--- Tour 1 ---
Merlin lance une boule de feu (mana 40).
  Ogre subit 15 dégâts (285/300 PV)
Ogre frappe Merlin.
  Merlin subit 18 dégâts (52/70 PV)
--- Tour 2 ---
Merlin lance une boule de feu (mana 30).
  Ogre subit 15 dégâts (270/300 PV)
Ogre frappe Merlin.
  Merlin subit 18 dégâts (34/70 PV)
--- Tour 3 ---
Merlin lance une nuée toxique (mana 15).
  Ogre est empoisonné (3 PV par tour, 3 tours).
  Poison : Ogre perd 3 PV (267/300)
Ogre rugit !
  Merlin est étourdi.
  Ogre gagne un bouclier de 20.
--- Tour 4 ---
  Merlin est étourdi et passe son tour.
  Poison : Ogre perd 3 PV (264/300)
Ogre frappe Merlin.
  Merlin subit 18 dégâts (16/70 PV)
Pas de vainqueur après 4 tours.
```

Indices

Lisez bien l'**ordre** des étapes d'un tour : effets au début, puis choix entre attaque et compétence, puis coups reçus. Une **seule méthode** de début de tour peut porter tous les effets. Le boss est une **nouvelle classe fille** : si l'arène a besoin d'une modification pour l'accepter, relisez la version v0.3.

8. Les bonus du sujet

Badge	Défi
Abstrait	Rendez la classe mère **abstraite** avec deux méthodes abstraites : l'attaque et la compétence spéciale. Un « combattant générique » n'existe pas dans le jeu : que répond Java si vous écrivez `new Combattant(...)` ? Justifiez votre choix dans le rapport.

Badge	Défi
Résilient	Créez une **exception personnalisée** (par exemple `CibleKOException`) levée quand on attaque un combattant déjà K.O., et interceptée par le menu.
Archiviste	Sauvegardez l'équipe dans un fichier texte ou CSV (une ligne par combattant : type, nom, PV, ressource) et rechargez-la au démarrage.

*Idée supplémentaire, sans badge : un **coup critique** tiré au hasard. Attention : un test qui donne un résultat différent à chaque lancement est un mauvais test. Cherchez comment rendre le hasard reproductible.*

9. Les pièges fréquents de ce sujet

- **Des dégâts négatifs qui soignent** : un coup plus faible que l'armure donne un nombre négatif ; sans le minimum de 1, le combattant gagne des PV.
- **Soigner un K.O.** ou **frapper un K.O.** : oublier de vérifier l'état avant d'agir.
- **Un instanceof dans l'arène** : le polymorphisme est contourné ; l'arrivée du boss obligerait à la modifier.
- **Recopier le calcul des dégâts dans chaque classe fille** : si vous copiez, c'est que la classe mère manque.
- **Le mana ou les flèches qui passent sous 0** : vérifier la ressource avant de la dépenser.
- **Le poison appliqué à un combattant déjà K.O.**, ou le poison qui tient compte de l'armure.
- **Oublier super(...)** dans le constructeur d'une fille, ou le placer après une autre instruction.

10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi n'y a-t-il pas de setter pour les PV ? Quelles règles sont ainsi protégées ?
- Quelle duplication de code la classe mère a-t-elle évitée entre guerrier, mage et archer ?
- Quelle est la ligne de code où le **type réel** d'un combattant décide de son attaque ou de sa compétence ?
- Que faudrait-il modifier pour ajouter un paladin ? Combien de classes existantes ?
- Comment votre code garantit-il qu'un effet d'état ne s'applique jamais à un combattant K.O. ?

11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter un **paladin** (110 PV, armure 10, attaque de base 14, compétence : se soigner de 20 PV) et le voir combattre sans toucher à l'arène.
- Passer le minimum de dégâts de 1 à 2.
- Ajouter un effet **saignement** (comme le poison, mais qui s'arrête si le combattant est soigné).
- Faire utiliser la compétence spéciale tous les 2 tours au lieu de 3.

Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

Relation	Sens
Guerrier, Mage, Archer, MonstreBoss → Combattant	Héritage : chaque type **est un** combattant.
Arene → Combattant (deux à la fois)	L'arène **manipule** des combattants par le type commun, sans connaître leur type réel.
Demo, Menu → Arene	Les classes de test et de menu **utilisent** l'arène.